

Tegaki 計画・設計ドキュメント体系リファクタリング提案

1. 提案の目的

Tegakiプロジェクトでは、長期構想、未実装機能、技術設計、現在の進捗、Phaseごとの実装計画などを複数のMarkdownファイルで管理している。

現在の運用は、概ね次の流れになっている。

1. 思いついた機能や構想を、比較的自由な形で記録する
2. AIとの対話を通して、構想を計画書や設計書に整理する
3. 実際に着手する段階で、必要な内容を `PhaseX.md` へ抽出する
4. `PhaseX.md` を実装作業の基準として使用する

この流れ自体は維持したい。

一方で、計画、現在の状況、確定済みの設計、検討中の案、作業用Phase文書の境界が曖昧になると、以下の問題が起こりやすい。

- 同じ情報が複数文書に重複する
- 古い計画が現在も有効なのか判断しにくい
- アイデア段階の案と実装決定済みの仕様が混在する
- AIが、検討中の案を確定事項として扱う
- Phase完了後も一時的な実装手順が正本として残る
- ファイル移動や改名によってリンク切れが発生する
- 新しいAIセッションが、どの文書から読むべきか判断しにくい

そこで、既存文書を全面的に廃棄するのではなく、各文書の責任範囲と、情報が移動する経路を整理したい。

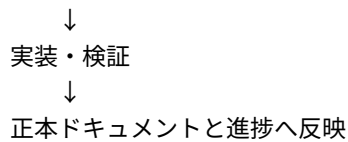
2. 基本方針

今回の整理では、次の原則を採用する。

2.1 既存の開発フローを維持する

以下の流れは変更しない。

```
graph TD; A[思いつき・雑案] --> B[検討・計画書化]; B --> C[採用判断]; C --> D[PhaseX.mdへ実装範囲を抽出];
```



「思いついた内容をすぐ正式計画へ昇格させないこと」と、「実装時にはPhase文書へ必要事項を抽出すること」を重視する。

2.2 一つの情報につき、原則として正本を一つにする

同じ内容を複数ファイルへ詳細に複製しない。

別文書から参照する場合は、要約とリンクのみを記載する。

2.3 文書名だけでなく、文書の役割を明文化する

各文書の先頭に、少なくとも以下を記載する。

- > 文書の役割:
- > 対象:
- > この文書に書くもの:
- > この文書に書かないもの:
- > 更新するタイミング:
- > 正本となる情報:

2.4 アイデア、提案、決定、実装を明確に区別する

最低でも次の状態を区別する。

- **IDEA** : 思いつき、未整理
- **DRAFT** : 整理中、検討中
- **PROPOSED** : 実装候補として提案済み
- **ACCEPTED** : 方針として採用
- **PLANNED** : Phaseへの抽出待ち、または抽出済み
- **IN_PROGRESS** : 実装中
- **IMPLEMENTED** : 実装済み
- **DEFERRED** : 保留
- **REJECTED** : 不採用
- **SUPERSEDED** : 別案に置き換え済み

状態名は、既存の用語との整合性を確認したうえで調整してよい。

3. 提案するドキュメント構成

既存の **AGENTS.md**、**TEGAKI.md**、**PROGRESS.md**、**ARCHITECTURE.md** などを尊重し、役割を整理する。

```
/
├── AGENTS.md
├── TEGAKI.md
├── tegaki_work/
│   ├── PROGRESS.md
│   ├── ARCHITECTURE.md
│   ├── PHASE4Z_BOUNDARY.md
│   └── phases/
│       ├── PhaseX.md
│       └── ...
├── 開発用資料保管庫/
│   ├── BRAINSTORM.md
│   ├── PROJECT_PLAN.md
│   ├── DECISIONS.md
│   ├── proposals/
│   │   ├── 00_計画索引.md
│   │   ├── 01_短中期ロードマップ.md
│   │   ├── 個別提案書.md
│   │   └── ...
│   └── archive/
│       ├── completed-phases/
│       ├── superseded-proposals/
│       └── legacy-documents/
```

実際のディレクトリ名や配置は、既存リンクへの影響を調査してから決定する。

4. 各文書の役割

4.1 AGENTS.md

役割

Codex、Claude Code、ChatGPTなど、プロジェクトを扱うAI全体に共通する作業規則。

記載対象

- 作業開始時に読む文書
- 禁止事項
- Git運用
- コード編集時のルール
- テスト・検証方法
- 文書更新の義務
- Phase文書の扱い
- AIが推測で仕様を確定してはいけない範囲

記載しないもの

- 個別機能の詳細設計
 - 日々の進捗
 - 一時的な作業メモ
 - 未整理のアイデア
-

4.2 TEGAKI.md

役割

Tegakiプロジェクトの恒久的な技術方針、原則、禁止事項の正本。

記載対象

- プロジェクト全体で守る原則
- 技術的制約
- 凍結中の技術
- 互換性方針
- 変更に明示的な承認が必要な事項
- 将来のPhaseでも維持する設計上の境界

記載しないもの

- 未採用のアイデア
 - 一時的な作業手順
 - 個別Phaseのタスクリスト
 - 単なる実装履歴
-

4.3 PROJECT_PLAN.md

役割

プロジェクトが最終的に何を目指し、どの段階を経て発展するかを示す長期計画。

記載対象

- Tegakiの目的
- 想定する利用者と利用場面
- お絵かきツールからアニメ制作ツールへの発展方針
- 長期的な完成イメージ
- 大分類としての開発段階
- 各段階の到達条件
- 現時点で対象外とする事項
- 将来候補となる大規模機能へのリンク

記載しないもの

- 日々の進捗

- 実装中の細かなタスク
- 未整理の思いつき
- Phase固有の実装手順
- 個別ソースコードの変更内容

性質

比較的更新頻度の低い文書とする。

短中期ロードマップとの重複が大きい場合は、以下のいずれかに整理する。

1. `PROJECT_PLAN.md` を長期構想、`01_短中期ロードマップ.md` を直近の優先順位とする
2. 既存のロードマップを `PROJECT_PLAN.md` に統合する
3. `PROJECT_PLAN.md` を新設せず、既存ロードマップの役割を明文化する

Codexは、重複状況を調査したうえで推奨案を提示すること。

4.4 BRAINSTORM.md

役割

ユーザーが思いついた内容を、整理や採否判断の前に自由に投入する場所。

この文書は、正式仕様書や実装指示書ではない。

記載対象

- 思いついた機能
- UI案
- 操作方法の案
- 将来的な発展案
- 技術的に可能か不明な案
- 既存案への疑問
- 問題意識
- 他ツールを使って感じた不満
- AIに後で整理させたい断片的メモ

記載しないもの

- 確定済みの技術方針
- 実装中タスクの進捗
- 正式な完了条件
- 現在のアーキテクチャの説明
- AIが必ず従うべき命令

運用原則

`BRAINSTORM.md` 内の内容は、明示的に昇格されるまで実装対象としない。

AIは、この文書に書かれているという理由だけで、機能を実装したり既存設計を変更したりしてはならない。

推奨フォーマット

完全な文章でなくてもよい。

未整理アイデア

IDEA-0001

- 日付: YYYY-MM-DD
- 状態: IDEA
- 内容:
 - レイヤーを直接動かして簡易アニメを作れないか
 - Live2Dよりアプリ内で描きやすくしたい
- 動機:
 - 外部ツールとの往復を減らしたい
- 疑問:
 - ラスター変形とメッシュ変形の境界をどうするか
- 関連候補:
 - メッシュ
 - CAF
 - タイムライン

入力の負担を増やさないため、ID、日付、分類などはAIが後から付与してもよい。

ユーザーは箇条書きや断片的な文章をそのまま追記できるものとする。

既存メモとの関係

現在の「実装したいことメモ」が同じ役割を持っている場合は、以下を検討する。

- 内容を維持したまま `BRAINSTORM.md` 相当として扱う
- Markdownへ変換する
- 既存ファイル名を残し、役割だけ明文化する
- 新しい文書へ移行し、旧ファイルから案内リンクを設置する

リンク切れを避けるため、即時削除や単純な改名は行わない。

4.5 `proposals/00_計画索引.md`

役割

正式な検討対象となった提案書を一覧化する索引。

記載対象

- 提案書名
- 状態
- 優先度
- 関連する長期目標

- 関連するPhase
- 依存関係
- 採用・保留・却下の状態
- 文書へのリンク

記載例

ID	提案	状態	優先度	Phase	備考
---	---	---	---	---	---
P-015	ボーン・CAF階層パーツ	PROPOSED	中	未定	境界設計の確認が必要
P-016	モーションパフォーム記録	DRAFT	中	未定	タイムライン依存
P-017	自動メッシュ	DRAFT	低	未定	WebGL系方針との確認が必要

注意

索引は提案内容の正本にしない。

詳細は個別提案書に置き、索引では状態と参照先のみを管理する。

4.6 個別提案書

役割

`BRAINSTORM.md` から昇格したアイデアを、実装候補として検討できる形へ整理する。

推奨構成

提案名

状態

DRAFT / PROPOSED / ACCEPTED / DEFERRED / REJECTED

背景

解決したい問題

利用者から見た価値

想定する動作

対象範囲

対象外

既存システムへの影響

技術的な論点

未決事項

リスク

依存関係

Phaseへ抽出する条件

関連文書

提案書の段階では、実装ファイルや具体的な関数単位まで固定しすぎない。

実際のコード構造を確認しないまま、詳細実装を正本化しないこと。

4.7 DECISIONS.md

役割

採用済みの設計判断と、その理由を記録する。

Gitの変更履歴ではなく、「なぜその方式を選んだのか」を残す文書とする。

記載対象

- 複数案から一つを採用した判断
- 将来のAIが変更しそうな重要方針
- 一見すると不自然だが理由のある制約
- 却下案と却下理由
- 判断を見直す条件

推奨フォーマット

DEC-XXXX: 判断名

- 日付:
- 状態: ACCEPTED / SUPERSEDED
- 関連提案:
- 関連Phase:

決定

理由

検討した代替案

却下理由

影響範囲

見直し条件

注意

すべての小さな実装判断を記録する必要はない。

プロジェクト全体、複数機能、将来の拡張性に影響する判断を対象とする。

4.8 ARCHITECTURE.md

役割

現在実装されているシステム構造の全体像を示す。

記載対象

- 主要モジュール
- データモデル
- モジュール間の依存関係
- イベントの流れ
- 描画、レイヤー、履歴、保存、アニメーションなどの境界
- 重要な不変条件
- 変更時に影響が広がる箇所
- 詳細な境界文書へのリンク

記載しないもの

- 未採用案を現在の構造として記述すること
- Phase固有の作業手順
- 日々の進捗
- 単純なファイル一覧だけの説明

将来計画を書く場合は、「現在」と「将来候補」を明確に分離する。

4.9 PROGRESS.md

役割

プロジェクトの現在地を短時間で把握するための最新状況。

記載対象

- 現在の開発段階
- 実装済み機能
- 部分的に実装済みの機能
- 既知の不具合
- 現在進行中のPhase

- 次の候補
- テスト状況
- 直近の重要な変更
- 詳細文書へのリンク

記載しないもの

- 長期構想の詳細
- 個別機能の完全な仕様
- 長大な作業ログ
- 過去の進捗をすべて追記した履歴

過去の詳細はGit履歴または完了Phase文書へ残す。

4.10 PhaseX.md

役割

採用済みの計画から、今回実装する範囲だけを抽出した実行用文書。

Phase文書に含めるもの

- Phaseの目的
- 前提条件
- 対象範囲
- 対象外
- 実装項目
- 変更予定箇所
- 完了条件
- テスト方法
- 回帰確認項目
- 禁止事項
- 関連する提案、決定、アーキテクチャ
- 未解決事項
- 作業終了時に更新する正本文書

Phase文書に含めないもの

- 長期計画全体の複製
- 関係のない将来案
- 未採用のプレスト
- 既存文書からコピーした大量の重複情報

Phase完了時の処理

Phase完了時には、以下を確認する。

1. 実装結果を `PROGRESS.md` に反映したか
2. 現在構造の変更を `ARCHITECTURE.md` に反映したか
3. 重要な判断を `DECISIONS.md` に反映したか
4. 提案索引の状態を更新したか

5. 未完了項目を次Phaseまたは提案書へ戻したか
6. Phase文書を完了済みとして保存したか
7. リンク切れがないか検査したか

5. 情報の昇格フロー

情報は以下のように昇格させる。

5.1 ブレストから提案書

次の条件を満たした内容を、個別提案書へ整理する。

- ・解決したい問題がある程度明確
- ・プロジェクトの方向性と大きく矛盾しない
- ・単なる思いつき以上の検討価値がある
- ・他の機能との関係を調査する必要がある
- ・ユーザーが整理を希望した

ブレストから削除する必要はない。

昇格後は、元の項目に次のような参照を追加する。

- 状態: PROMOTED
- 昇格先: proposals/XX_提案名.md

5.2 提案書から正式計画

提案が採用された場合は、状態を `ACCEPTED` に変更する。

長期計画へ影響する場合のみ、`PROJECT_PLAN.md` またはロードマップへ要約を反映する。

5.3 正式計画からPhase

実装開始時には、提案書全体をそのままPhaseへコピーしない。

今回実装する範囲、完了条件、検証方法を抽出し、現在のコードを確認したうえで `PhaseX.md` を作成する。

5.4 Phaseから正本文書

実装完了後、実装結果を以下へ戻す。

- | | |
|-------|--------------------------------|
| 実装状況 | → <code>PROGRESS.md</code> |
| 現在の構造 | → <code>ARCHITECTURE.md</code> |
| 重要な判断 | → <code>DECISIONS.md</code> |
| 提案の状態 | → 計画索引・個別提案書 |

恒久的な原則	→ TEGAKI.md
AI作業規則	→ AGENTS.md

6. AI_HANDOFF.md について

現時点では、独立した恒久文書として必須にはしない。

CodexとClaude Codeの交代が増え、未コミット作業や途中経過の申し送りが必要になった場合に導入する。

導入する場合は、一時的な申し送り専用とする。

AI Handoff

- 更新日時:
- 作業ブランチ:
- 作業担当:
- 関連Phase:

今回行ったこと

未完了

変更したファイル

実行したテスト

確認してほしい点

次の候補

作業ツリーの注意

永続的な設計情報を AI_HANDOFF.md に置かない。

恒久情報は、必ず適切な正本文書へ反映する。

7. リンク切れへの対応

現在、一部のリンクが切れている可能性があるため、文書移動の前にリンク監査を行う。

調査対象

- Markdown内の相対リンク
- GitHub Raw URL
- URLエンコードされた日本語ファイル名

- 改名済みファイルへのリンク
- 削除済みファイルへのリンク
- 見出しアンカー
- `GitHubURL` ナビゲーション内のリンク
- `AGENTS.md` などから参照されている必読文書

原則

- リンク調査前にファイルを削除しない
- 改名は必要最小限にする
- 既存URLを壊す場合は旧ファイルを案内用に残す
- 単純なリンク切れ修正と文書再編を同一コミットに混在させない
- 日本語ファイル名を英語名へ一括変更しない
- 外部AI用Raw URLナビゲーションを最後に更新する

旧ファイルを残す場合の例

この文書は移動しました

現在の正本:

[新しい文書名](相対リンク)

このファイルは既存リンク維持のために残しています。
新規情報は追記しないでください。

8. Codexへ依頼したい作業

この提案を受けて、直ちにファイル移動や大量編集を行わないこと。

まず読み取り専用で調査し、以下の結果を提示してほしい。

第1段階：文書の棚卸し

1. 計画、提案、進捗、設計、Phase、メモに該当するファイルを列挙する
2. 各文書の現在の役割を要約する
3. 内容が重複している文書を示す
4. 役割が曖昧な文書を示す
5. 古い可能性のある文書を示す
6. 他文書から参照されている文書を示す
7. リンク切れまたはリンク切れの可能性を示す
8. 正本が複数存在する可能性のある情報を示す

第2段階：移行案の作成

以下を含む具体的な移行案を提示する。

- 維持するファイル
- 役割だけ変更するファイル
- 統合候補

- 分割候補
- 新設候補
- アーカイブ候補
- 改名候補
- 改名によるリンク影響
- 推奨ディレクトリ構成
- 移行順序
- ロールバック方法

第3段階：最小変更案と整理優先案の比較

少なくとも次の2案を提示する。

A. 最小変更案

- 既存ファイル名と場所を極力維持
- 各文書の冒頭へ役割説明を追加
- `BRAINSTORM.md` 相当だけを新設
- 索引とリンクを修正
- 重複を段階的に解消

B. 整理優先案

- 長期計画、プレスト、決定記録、提案、Phase、アーカイブを明確に分離
- 必要に応じてファイル移動や改名を実施
- Raw URLナビゲーションを再構築
- 旧リンク維持用の案内文書を配置

各案について、利点、欠点、作業量、リンク切れリスク、AIによる読み違いの可能性を比較する。

9. Codexに対する制約

調査および提案時には、以下を守ること。

- ソースコードを変更しない
- 計画書を削除しない
- ファイルを移動しない
- ファイルを改名しない
- 未採用案を正式仕様として扱わない
- 文書間の矛盾を推測だけで解消しない
- `TEGAKI.md` の規定を勝手に変更しない
- 凍結中の技術を有効化しない
- 現在のPhaseの範囲を拡大しない
- リンク切れを確認せずにURLを書き換えない
- 判断が必要な箇所は、選択肢と影響を提示してユーザーへ確認する

10. Codexに求める最初の回答形式

最初の回答は、次の形式にする。

Tegaki ドキュメント体系調査結果

1. 調査した文書

2. 現在の文書体系

3. 重複している情報

4. 役割が曖昧な文書

5. リンク切れ・参照上の問題

6. 既存運用の長所

7. 最小変更案

8. 整理優先案

9. 推奨案

10. 移行手順案

11. ユーザーの判断が必要な項目

文書の内容を確認できなかった場合は、確認できなかったことを明記する。

存在しない文書の内容を推測しない。

11. 今回の整理で達成したい状態

最終的には、各文書が次の質問に一つずつ答える状態を目指す。

文書	答える質問
AGENTS.md	AIはどのように作業するか
TEGAKI.md	絶対に守る技術方針は何か
PROJECT_PLAN.md	最終的に何を作るのか
BRAINSTORM.md	どのようなアイデアが出ているか
計画索引	どの提案が存在し、どの状態か
個別提案書	その案は何を解決し、どう実現したいか
DECISIONS.md	なぜその設計を選んだのか
ARCHITECTURE.md	現在どのような構造になっているか
PROGRESS.md	今どこまでできているか

文書	答える質問
PhaseX.md	今回は何を、どこまで実装するか
AI_HANDOFF.md	次の作業者が直ちに知るべきことは何か

重要なのは、文書数を増やすことではない。

ユーザーが自由に発想を投入でき、AIがそれを勝手に正式仕様と誤認せず、実装時には必要な内容だけをPhaseへ抽出し、完了後は正しい文書へ結果を戻せる情報循環を作ることが目的である。